



Analyzing

Online Shopping
Platform



By - Drishti Khosla

SQL Assignment

ADVANCED

Finally, we have expanded our understanding of SQL queries. Now, we will learn some more advanced queries and cover the crucial concepts using them.

So, for the business purpose, we have expanded our data-tables, and now we will work with 6 tables, which are look like:

1.Customers (customer_id, name, city, signup_date, email)

2.Orders (order_id, customer_id, order_date, total_amount, payment_mode, shipper_id)

3.Products (product_id, product_name, category, price, stock_qty, supplier_id)

4.Order_Items (order_item_id, order_id, product_id, quantity, price_each)

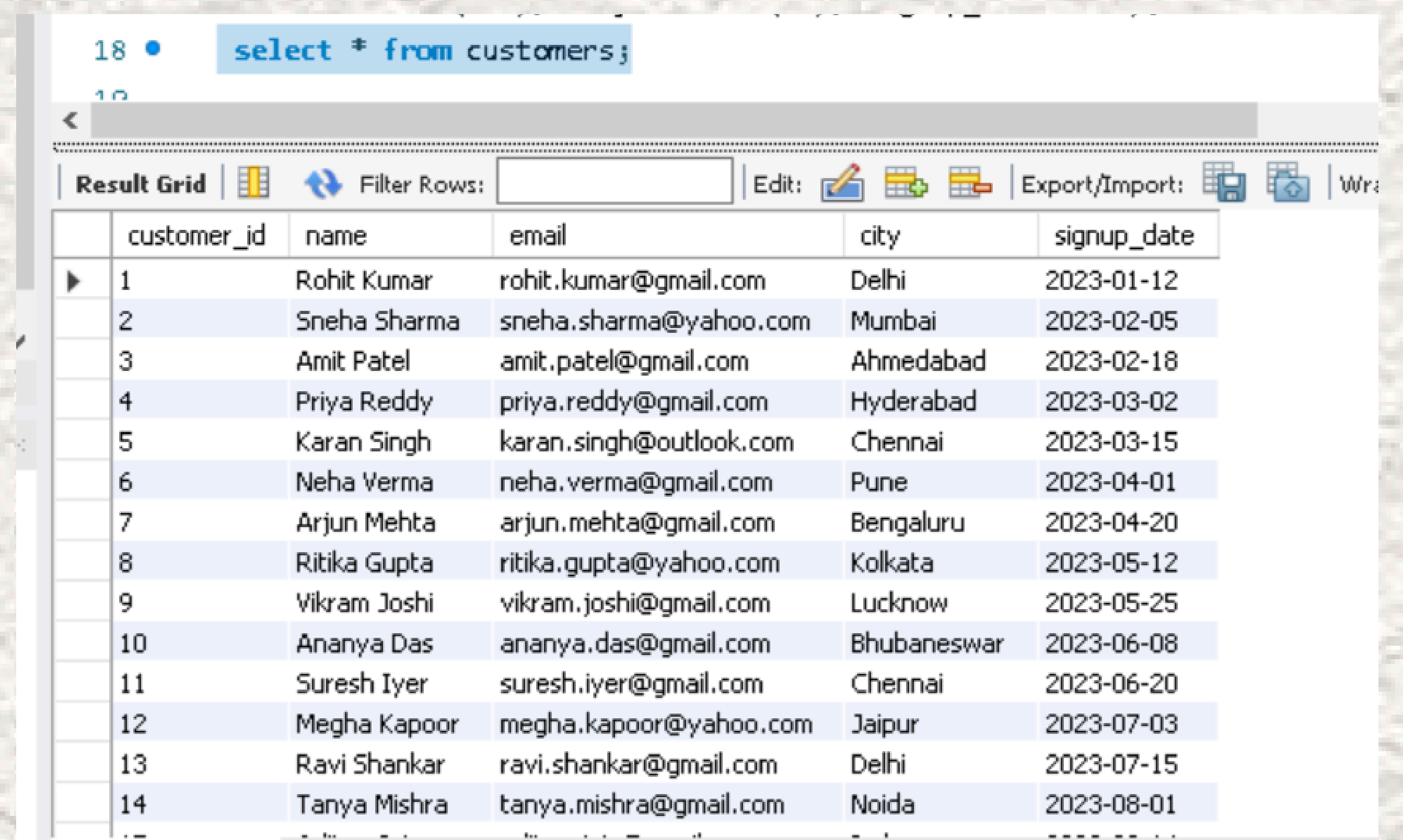
5.Suppliers (supplier_id, supplier_name, contact, city)

6.Shippers (shipper_id, shipper_name, contact)

7.Payments (payment_id, order_id, amount, payment_date, payment_mode)

Created the database RetailDB_3 and created the following tables :-

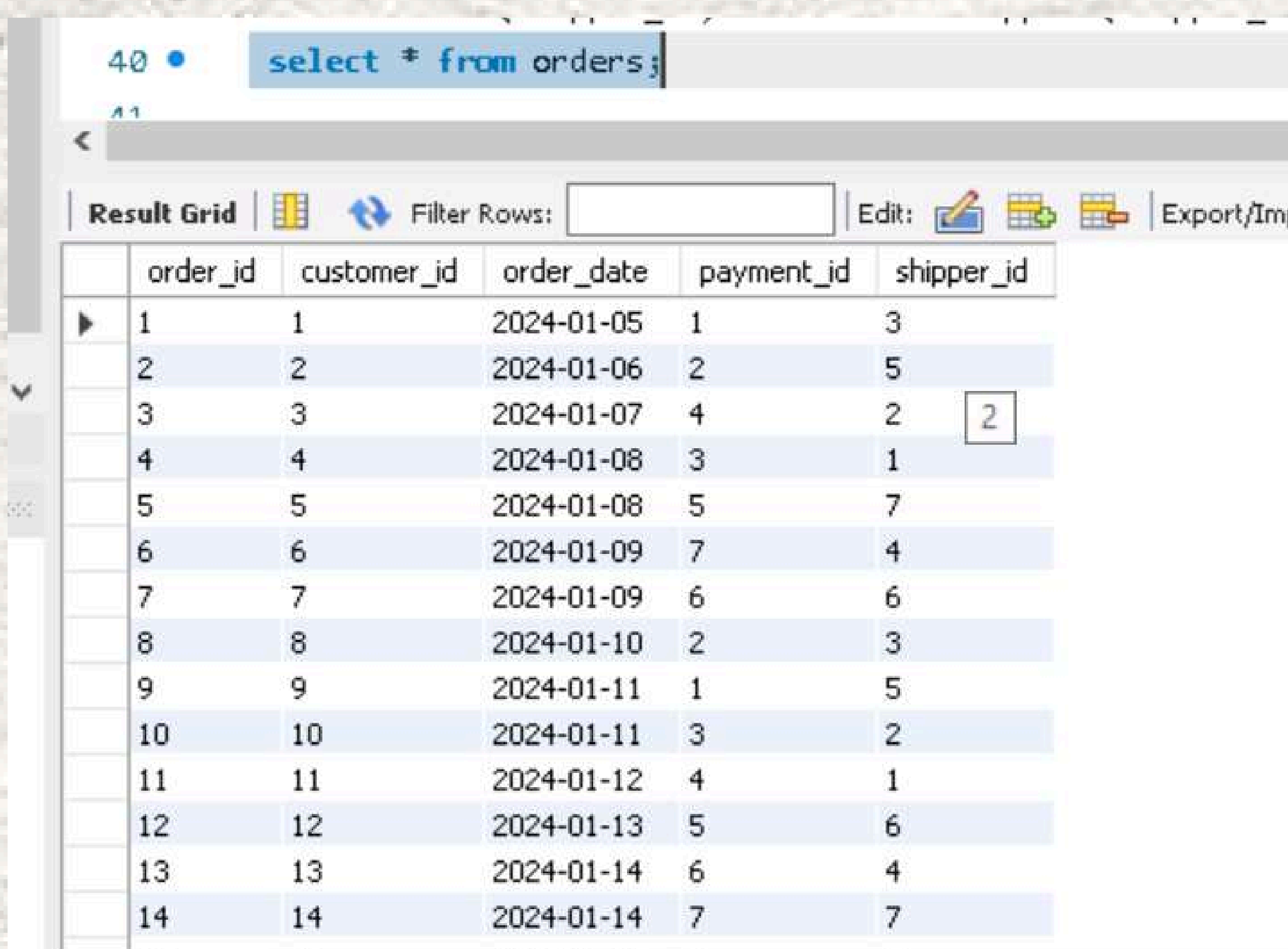
This is how
the Customer
table looked
after
importing
the data



The screenshot shows a database query window with the SQL statement `select * from customers;` and a result grid displaying 14 rows of customer data. The columns are customer_id, name, email, city, and signup_date.

customer_id	name	email	city	signup_date
1	Rohit Kumar	rohit.kumar@gmail.com	Delhi	2023-01-12
2	Sneha Sharma	sneha.sharma@yahoo.com	Mumbai	2023-02-05
3	Amit Patel	amit.patel@gmail.com	Ahmedabad	2023-02-18
4	Priya Reddy	priya.reddy@gmail.com	Hyderabad	2023-03-02
5	Karan Singh	karan.singh@outlook.com	Chennai	2023-03-15
6	Neha Verma	neha.verma@gmail.com	Pune	2023-04-01
7	Arjun Mehta	arjun.mehta@gmail.com	Bengaluru	2023-04-20
8	Ritika Gupta	ritika.gupta@yahoo.com	Kolkata	2023-05-12
9	Vikram Joshi	vikram.joshi@gmail.com	Lucknow	2023-05-25
10	Ananya Das	ananya.das@gmail.com	Bhubaneswar	2023-06-08
11	Suresh Iyer	suresh.iyer@gmail.com	Chennai	2023-06-20
12	Megha Kapoor	megha.kapoor@yahoo.com	Jaipur	2023-07-03
13	Ravi Shankar	ravi.shankar@gmail.com	Delhi	2023-07-15
14	Tanya Mishra	tanya.mishra@gmail.com	Noida	2023-08-01

This is how
the Orders
table looked
after
importing
the data



The screenshot shows a database query window with the SQL statement `select * from orders;` and a result grid displaying 14 rows of order data. The columns are order_id, customer_id, order_date, payment_id, and shipper_id.

order_id	customer_id	order_date	payment_id	shipper_id
1	1	2024-01-05	1	3
2	2	2024-01-06	2	5
3	3	2024-01-07	4	2
4	4	2024-01-08	3	1
5	5	2024-01-08	5	7
6	6	2024-01-09	7	4
7	7	2024-01-09	6	6
8	8	2024-01-10	2	3
9	9	2024-01-11	1	5
10	10	2024-01-11	3	2
11	11	2024-01-12	4	1
12	12	2024-01-13	5	6
13	13	2024-01-14	6	4
14	14	2024-01-14	7	7

This is how
the Products
table looked
after
importing
the data

```
34
35 • select * from products;
36
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wra

	product_id	product_name	category	price	stock_qty	supplier_id
▶	1	Iphone 15	Electronics	79999.00	50	1
	2	Samsung Galaxy	Electronics	74999.00	40	2
	3	Noise Smartwatch	Wearables	4999.00	100	3
	4	Boat Earbuds	Wearables	2499.00	200	4
	5	Kurta Set	Fashion	1999.00	150	7
	6	Running Shoes	Fashion	2999.00	120	7
	7	Prestige Mixer Grinder	Home Appliances	6499.00	80	5
	8	Sony Bravia 55" TV	Electronics	59999.00	30	6
	9	Lenovo Laptop	Electronics	55999.00	45	2
	10	Philips Trimmer	Personal Care	1499.00	90	8
*	NULL	NULL	NULL	NULL	NULL	NULL

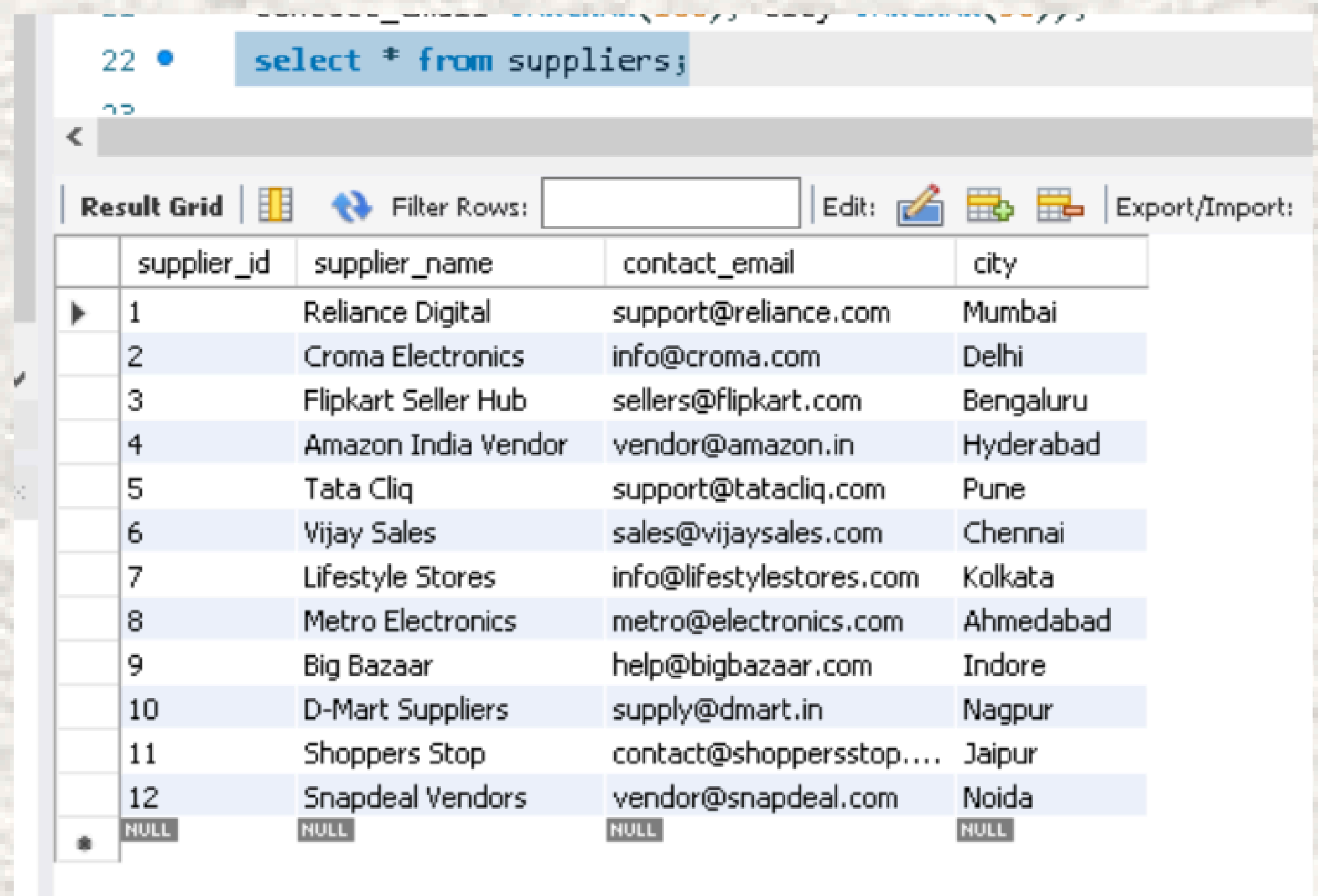
This is how
the
Orders_items
table looked
after
importing
the data

```
45 • select * from order_items;
46
```

Result Grid | Filter Rows: | Edit: | E

	order_item_id	order_id	product_id	quantity	price_each
▶	1	1	1	2	15000.00
	2	2	3	1	12000.00
	3	3	5	3	2500.00
	4	4	2	1	22000.00
	5	5	7	2	1800.00
	6	6	4	1	8000.00
	7	7	6	4	1500.00
	8	8	8	2	3500.00
	9	9	10	1	7000.00
	10	10	9	3	1200.00
	11	11	2	1	22000.00
	12	12	1	2	15000.00
	13	13	4	3	8000.00
	14	14	7	1	1800.00
	--	--	--	--	----

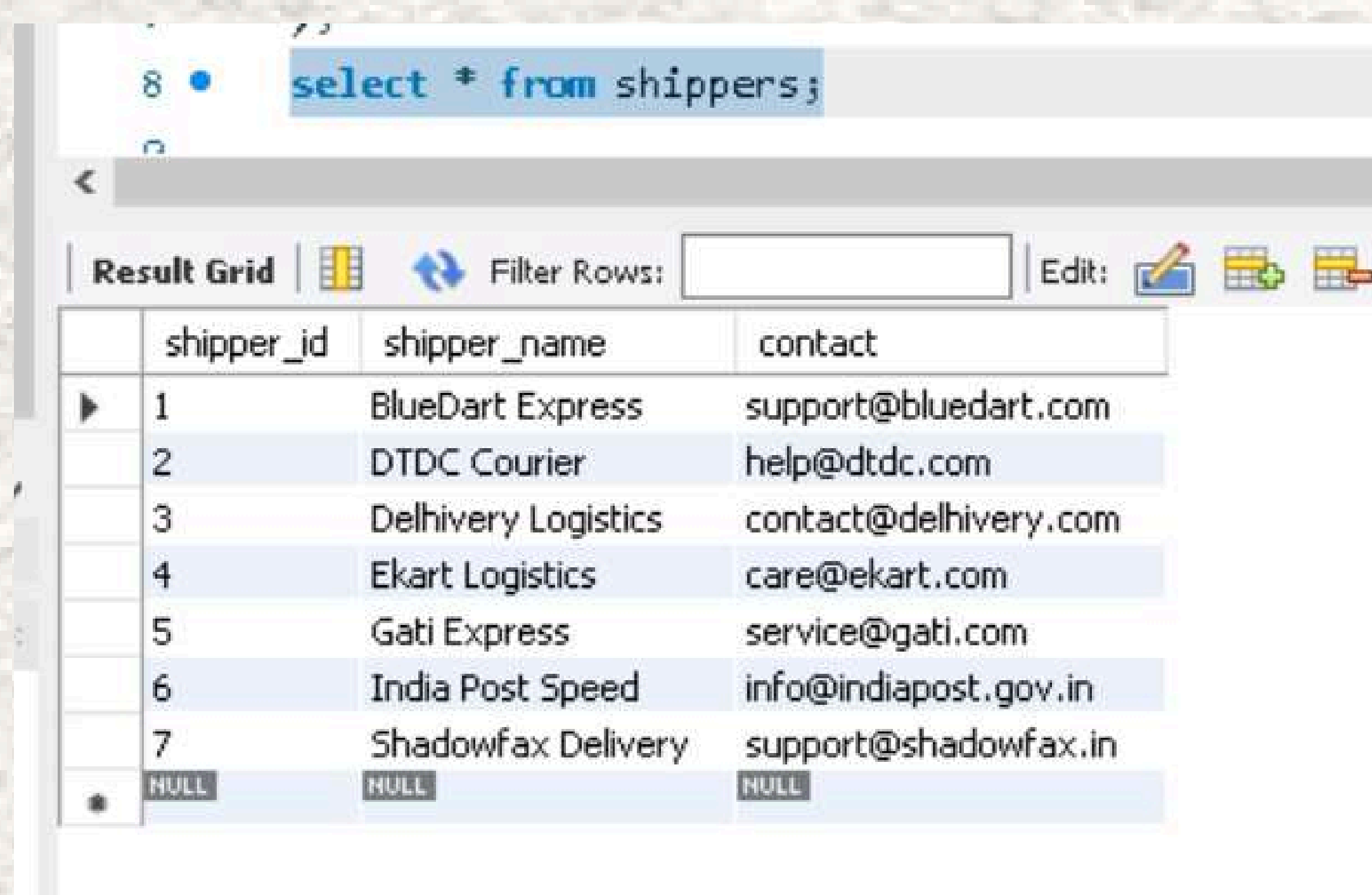
This is how
the Suppliers
table looked
after
importing
the data



The screenshot shows a database query interface with the SQL statement `select * from suppliers;` entered in the query editor. Below the editor, the 'Result Grid' displays the data for the Suppliers table. The table has four columns: `supplier_id`, `supplier_name`, `contact_email`, and `city`. There are 12 rows of data, each representing a different supplier. The last row shows a NULL value for the `supplier_id` column.

	supplier_id	supplier_name	contact_email	city
▶	1	Reliance Digital	support@reliance.com	Mumbai
	2	Croma Electronics	info@croma.com	Delhi
	3	Flipkart Seller Hub	sellers@flipkart.com	Bengaluru
	4	Amazon India Vendor	vendor@amazon.in	Hyderabad
	5	Tata Cliq	support@tatacliq.com	Pune
	6	Vijay Sales	sales@vijaysales.com	Chennai
	7	Lifestyle Stores	info@lifestylestores.com	Kolkata
	8	Metro Electronics	metro@electronics.com	Ahmedabad
	9	Big Bazaar	help@bigbazaar.com	Indore
	10	D-Mart Suppliers	supply@dmart.in	Nagpur
	11	Shoppers Stop	contact@shoppersstop....	Jaipur
*	12	Snapdeal Vendors	vendor@snapdeal.com	Noida
	NULL	NULL	NULL	NULL

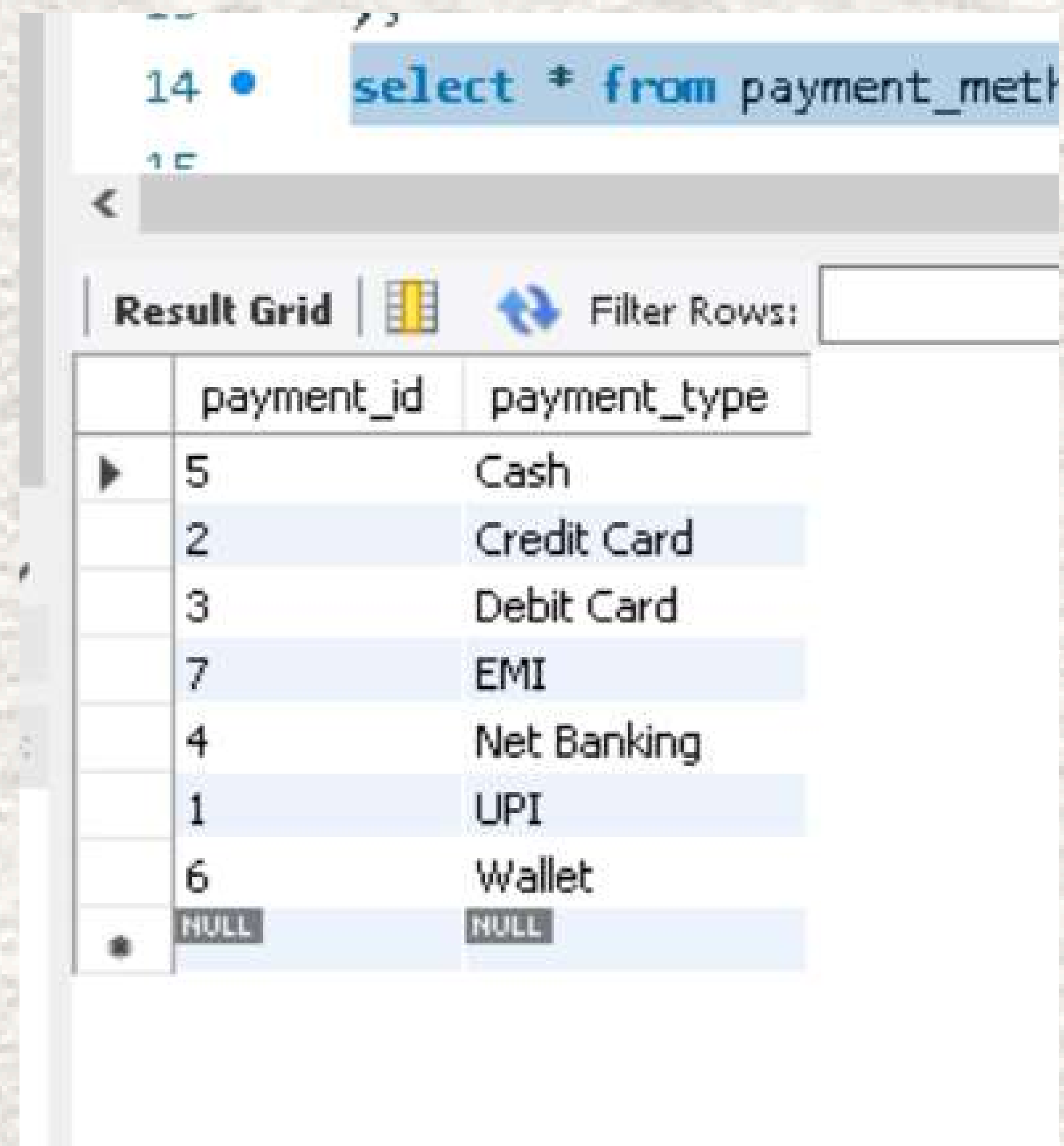
This is how
the Shippers
table looked
after
importing
the data



The screenshot shows a database query interface with the SQL statement `select * from shippers;` entered in the query editor. Below the editor, the 'Result Grid' displays the data for the Shippers table. The table has three columns: `shipper_id`, `shipper_name`, and `contact`. There are 7 rows of data, each representing a different shipper. The last row shows a NULL value for the `shipper_id` column.

	shipper_id	shipper_name	contact
▶	1	BlueDart Express	support@bluedart.com
	2	DTDC Courier	help@dtdc.com
	3	Delhivery Logistics	contact@delhivery.com
	4	Ekart Logistics	care@ekart.com
	5	Gati Express	service@gati.com
	6	India Post Speed	info@indiapost.gov.in
	7	Shadowfax Delivery	support@shadowfax.in
*	NULL	NULL	NULL

This is how
the Payments
table looked
after
importing
the data



The screenshot shows a database interface with a query editor at the top containing the SQL statement `select * from payment_method`. Below the editor is a 'Result Grid' section. The grid has two columns: 'payment_id' and 'payment_type'. It displays eight rows of data, including payment methods like Cash, Credit Card, Debit Card, EMI, Net Banking, UPI, and Wallet, as well as a row with NULL values. A 'Filter Rows' input field is located to the right of the grid header.

	payment_id	payment_type
▶	5	Cash
	2	Credit Card
	3	Debit Card
	7	EMI
	4	Net Banking
	1	UPI
	6	Wallet
★	NULL	NULL

Task: Solve all the below mentioned problems by writing the SQL queries.

a) Joins, Group By, Order By, Aggregations:

1. Find the total revenue collected by each shipper.
2. Show the top 5 highest-spending customers along with their total payments.
3. Find product categories where the average selling price is greater than 8000.
4. Show the total number of orders placed per city, sorted by highest to lowest.
5. Find suppliers who supply more than 1 product category.
6. Show each order along with the number of items it contains.

b) Subqueries – Nested & Correlated:

7. Find customers who have spent more than the average spending of all customers.
8. List all products whose price is higher than the average product price in their category.
9. Show customers who placed at least one order with total_amount greater than 50,000.
10. Find customers who placed more orders than the average number of orders per customer.
11. Find the most expensive product(s) in the catalog.

c) Window Functions:

12. Rank customers based on their total spending.
13. Find cumulative sales amount by order date.
14. Get each customer's order count and show their percentage contribution.
15. Show the most recent order per customer.
16. List each product with its sales quantity and rank products within each category by total sales.

d) CTE & Case:

- 17. Categorize products as 'High', 'Medium', or 'Low' price using CASE.
- 18. Use a CTE to find top 3 customers by spending.
- 19. Use a CTE with CASE to classify customers by loyalty (based on number of orders).
- 20. Find monthly revenue growth percentage compared to the previous month.
- 21. Find top 2 customers per city based on total spending.

e) Miscellaneous Advanced Joins & Aggregations

- 22. Find top 3 cities with highest sales revenue including shipper name.
- 23. List all orders with customer name, product name, supplier, and shipper.
- 24. Show total sales per supplier along with average order value.
- 25. Show product categories that contributed more than 30% of total sales revenue.

1. Find the total revenue collected by each shipper.

```
46
47 • select s.shipper_name, sum(oi.quantity * oi.price_each) as total_revenue
48 from orders o join shippers s on o.shipper_id = s.shipper_id
49 join order_items oi on o.order_id = oi.order_id
50 group by s.shipper_name;
51
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	shipper_name	total_revenue
▶	BlueDart Express	149700.00
	DTDC Courier	200200.00
	Delhivery Logistics	122400.00
	Ekart Logistics	123500.00
	Gati Express	105400.00
	India Post Speed	155100.00
	Shadowfax Delivery	100800.00

2. Show the top 5 highest-spending customers along with their total payments.

```
52 • select c.name, sum(oi.quantity * oi.price_each) as total_spent
53 from customers c join orders o on c.customer_id = o.customer_id
54 join order_items oi on o.order_id = oi.order_id
55 group by c.customer_id order by total_spent desc limit 5;
56
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows: |

	name	total_spent
▶	Rohit Kumar	152000.00
	Megha Kapoor	88500.00
	Sneha Sharma	83100.00
	Ritika Gupta	77600.00
	Tanya Mishra	75200.00

3. Find product categories where the average selling price is greater than 8000.

```
56
57 • select p.category, avg(oi.price_each) as avg_price
58 from products p join order_items oi on p.product_id = oi.product_id
59 group by p.category having avg(oi.price_each) > 8000;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
category	avg_price		
▶ Electronics	10963.333333		
Wearables	9866.666667		

4. Show the total number of orders placed per city, sorted by highest to lowest.

```
60
61 • select c.city, count(o.order_id) as total_orders
62 from customers c join orders o on c.customer_id = o.customer_id
63 group by c.city order by total_orders desc;
```

city	total_orders
▶ Delhi	10
Chennai	10
Mumbai	6
Bengaluru	6
Ahmedabad	5
Kolkata	5
Lucknow	5
Bhubaneswar	5
Jaipur	5
Noida	5
Indore	5
Hyderabad	4
Pune	4

5. Find suppliers who supply more than 1 product category.

```
64
65 • select s.supplier_name, count(distinct p.category) as category_count
66 from suppliers s join products p on s.supplier_id = p.supplier_id
67 group by s.supplier_id having count(distinct p.category) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

supplier_name	category_count
---------------	----------------

6. Show each order along with the number of items it contains.

```
68
69 • select o.order_id, count(oi.order_item_id) as item_count
70 from orders o join order_items oi on o.order_id = oi.order_id
71 group by o.order_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

order_id	item_count
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	1
13	1
14	1

Result 44 x

7. Find customers who have spent more than the average spending of all customers.

```
72
73 • SELECT name FROM Customers WHERE customer_id IN ( SELECT o.customer_id FROM Orders o
74 JOIN Order_Items oi ON o.order_id = oi.order_id GROUP BY o.customer_id HAVING SUM(oi.quantity * oi.price_each) >
75 (SELECT AVG(total_spent) FROM (SELECT SUM(quantity * price_each) AS total_spent FROM Orders o2
76 JOIN Order_Items oi2 ON o2.order_id = oi2.order_id GROUP BY o2.customer_id) t));
77
```

Result Grid

name
Rohit Kumar
Sneha Sharma
Amit Patel
Arjun Mehta
Ritika Gupta
Megha Kapoor
Tanya Mishra
Aditya Jain

8. List all products whose price is higher than the average product price in their category.

```
77
78 • SELECT product_name FROM Products p WHERE price >
79 ( SELECT AVG(price) FROM Products WHERE category = p.category);
80
```

Result Grid

product_name
Iphone 15
Samsung Galaxy
Noise Smartwatch
Running Shoes

9. Show customers who placed at least one order with total_amount greater than 50,000.

```
80
81 • select distinct c.name from customers c join orders o on c.customer_id = o.customer_id
82 join order_items oi on o.order_id = oi.order_id group by o.order_id, c.name having sum(oi.quantity * oi.price_each) > 50000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

name
Rohit Kumar

10. Find customers who placed more orders than the average number of orders per customer.

```
83
84 • select customer_id from orders group by customer_id having count(order_id) >
85 (select avg(order_count) from ( select count(order_id) as order_count from orders
86 group by customer_id) x);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id
2
7

11. Find the most expensive product(s) in the catalog

```
87
88 • select product_name, price from products where price = (select max(price) from products);
89
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	product_name	price
▶	Iphone 15	79999.00

12. Rank customers based on their total spending.

```
89
90 • select c.name, sum(oi.quantity * oi.price_each) as total_spent,
91 rank() over (order by sum(oi.quantity * oi.price_each) desc) as spending_rank
92 from customers c join orders o on c.customer_id = o.customer_id
93 join order_items oi on o.order_id = oi.order_id
94 group by c.customer_id, c.name;
95
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	name	total_spent	spending_rank
▶	Rohit Kumar	152000.00	1
	Megha Kapoor	88500.00	2
	Sneha Sharma	83100.00	3
	Ritika Gupta	77600.00	4
	Tanya Mishra	75200.00	5
	Aditya Jain	72000.00	6
	Amit Patel	70000.00	7
	Arjun Mehta	66000.00	8
	Ravi Shankar	55500.00	9
	Suresh Iyer	48400.00	10
	Neha Verma	46400.00	11
	Priya Reddy	42600.00	12
	Ananya Das	30400.00	13
	Karan Singh	24800.00	14

Result 50

13.Find cumulative sales amount by order date.

```
94 group by c.customer_id, c.name;
95
96 • select o.order_date, sum(oi.quantity * oi.price_each) as daily_sales,
97 sum(sum(oi.quantity * oi.price_each)) over (order by o.order_date) as cumulative_sales
98 from orders o join order_items oi on o.order_id = oi.order_id
99 group by o.order_date;
```

	order_date	daily_sales	cumulative_sales
▶	2024-01-05	30000.00	30000.00
	2024-01-06	12000.00	42000.00
	2024-01-07	7500.00	49500.00
	2024-01-08	25600.00	75100.00
	2024-01-09	14000.00	89100.00
	2024-01-10	7000.00	96100.00
	2024-01-11	10600.00	106700.00
	2024-01-12	22000.00	128700.00
	2024-01-13	30000.00	158700.00
	2024-01-14	25800.00	184500.00
	2024-01-15	29000.00	213500.00
	2024-01-16	4500.00	218000.00
	2024-01-17	8300.00	226300.00
	2024-01-18	14000.00	240300.00
			

14.Get each customer’s order count and show their percentage contribution.

```
100
101 • select customer_id,
102 count(order_id) as order_count,
103 count(order_id) * 100.0 / sum(count(order_id)) over () as percentage_contribution
104 from orders group by customer_id;
```

	customer_id	order_count	percentage_contribution
▶	1	5	6.66667
	2	6	8.00000
	3	5	6.66667
	4	4	5.33333
	5	5	6.66667
	6	4	5.33333
	7	6	8.00000
	8	5	6.66667
	9	5	6.66667
	10	5	6.66667
	11	5	6.66667
	12	5	6.66667
	13	5	6.66667
	14	5	6.66667
	..	-	

15.Show the most recent order per customer.

105

106

107

108

109

SELECT * FROM (SELECT o.*,
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS rn
FROM Orders o) t WHERE rn = 1;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	order_id	customer_id	order_date	payment_id	shipper_id	rn
▶	71	1	2024-02-21	2	7	1
	72	2	2024-02-21	3	6	1
	73	3	2024-02-22	4	1	1
	59	4	2024-02-13	2	5	1
	74	5	2024-02-23	6	5	1
	52	6	2024-02-09	3	7	1
	75	7	2024-02-23	7	4	1
	63	8	2024-02-15	5	2	1
	64	9	2024-02-16	4	7	1
	65	10	2024-02-16	2	6	1
	66	11	2024-02-17	3	1	1
	67	12	2024-02-18	6	5	1
	68	13	2024-02-18	1	4	1
	69	14	2024-02-19	7	3	1

16.List each product with its sales quantity and rank products within each category by total sales.

108

109

110

111

112

113

114

FROM Orders o) t WHERE rn = 1;

SELECT p.product_name, p.category, SUM(oi.quantity) AS total_qty,
RANK() OVER (PARTITION BY p.category ORDER BY SUM(oi.quantity) DESC) AS category_rank
FROM Products p JOIN Order_Items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.category;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	product_name	category	total_qty	category_rank
▶	Lenovo Laptop	Electronics	18	1
	Iphone 15	Electronics	16	2
	Samsung Galaxy	Electronics	13	3
	Sony Bravia 55" TV	Electronics	10	4
	Running Shoes	Fashion	22	1
	Kurta Set	Fashion	15	2
	Prestige Mixer Grinder	Home Appliances	15	1
	Philips Trimmer	Personal Care	11	1
	Boat Earbuds	Wearables	13	1
	Noise Smartwatch	Wearables	8	2

17. Categorize products as ‘High’, ‘Medium’, or ‘Low’ price using CASE.

```
114
115 • SELECT product_name, price,
116 CASE WHEN price > 10000 THEN 'High' WHEN price BETWEEN 5000 AND 10000 THEN 'Medium'
117 ELSE 'Low' END AS price_category FROM Products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

	product_name	price	price_category
▶	Iphone 15	79999.00	High
	Samsung Galaxy	74999.00	High
	Noise Smartwatch	4999.00	Low
	Boat Earbuds	2499.00	Low
	Kurta Set	1999.00	Low
	Running Shoes	2999.00	Low
	Prestige Mixer Grinder	6499.00	Medium
	Sony Bravia 55" TV	59999.00	High
	Lenovo Laptop	55999.00	High
	Philips Trimmer	1499.00	Low

18. Use a CTE to find top 3 customers by spending.

```
118
119 • WITH CustomerSpending AS ( SELECT c.name, SUM(oi.quantity * oi.price_each) AS total_spent
120 FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id
121 JOIN Order_Items oi ON o.order_id = oi.order_id GROUP BY c.customer_id, c.name)
122 SELECT * FROM CustomerSpending ORDER BY total_spent DESC LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

	name	total_spent
▶	Rohit Kumar	152000.00
	Megha Kapoor	88500.00
	Sneha Sharma	83100.00

19. Use a CTE with CASE to classify customers by loyalty (based on number of orders).

```
123
124 • WITH OrderCounts AS (SELECT customer_id, COUNT(order_id) AS total_orders
125 FROM Orders GROUP BY customer_id)
126 SELECT customer_id, CASE WHEN total_orders >= 10 THEN 'Highly Loyal'
127 WHEN total_orders BETWEEN 5 AND 9 THEN 'Moderately Loyal'
128 ELSE 'Low Loyalty' END AS loyalty_status
129 FROM OrderCounts;
```

Result Grid

customer_id	loyalty_status
1	Moderately Loyal
2	Moderately Loyal
3	Moderately Loyal
4	Low Loyalty
5	Moderately Loyal
6	Low Loyalty
7	Moderately Loyal
8	Moderately Loyal
9	Moderately Loyal
10	Moderately Loyal
11	Moderately Loyal
12	Moderately Loyal
13	Moderately Loyal
14	Moderately Loyal

20. Find monthly revenue growth percentage compared to the previous month.

```
130
131 • WITH MonthlySales AS ( SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month,
132 SUM(oi.quantity * oi.price_each) AS revenue FROM Orders o
133 JOIN Order_Items oi ON o.order_id = oi.order_id GROUP BY month)
134 SELECT month, revenue, LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
135 ((revenue - LAG(revenue) OVER (ORDER BY month)) / LAG(revenue) OVER (ORDER BY month)) * 100 AS growth_percentage
136 FROM MonthlySales;
```

Result Grid

month	revenue	prev_month_revenue	growth_percentage
2024-01	479300.00	NULL	NULL
2024-02	477800.00	479300.00	-0.312956

21. Find top 2 customers per city based on total spending.

```
137
138 • WITH CitySpending AS ( SELECT c.city, c.name,
139 SUM(oi.quantity * oi.price_each) AS total_spent, RANK() OVER (PARTITION BY c.city ORDER BY
140 SUM(oi.quantity * oi.price_each) DESC) AS rnk FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id
141 JOIN Order_Items oi ON o.order_id = oi.order_id GROUP BY c.city, c.name)
142 SELECT * FROM CitySpending WHERE rnk <= 2;
```

Result Grid

	city	name	total_spent	rnk
▶	Ahmedabad	Amit Patel	70000.00	1
	Bengaluru	Arjun Mehta	66000.00	1
	Bhubaneswar	Ananya Das	30400.00	1
	Chennai	Suresh Iyer	48400.00	1
	Chennai	Karan Singh	24800.00	2
	Delhi	Rohit Kumar	152000.00	1
	Delhi	Ravi Shankar	55500.00	2
	Hyderabad	Priya Reddy	42600.00	1
	Indore	Aditya Jain	72000.00	1
	Jaipur	Megha Kapoor	88500.00	1
	Kolkata	Ritika Gupta	77600.00	1
	Lucknow	Vikram Joshi	24600.00	1
	Mumbai	Sneha Sharma	83100.00	1
	Noida	Tanya Mishra	75200.00	1

Result 59

22. Find top 3 cities with highest sales revenue including shipper name.

```
143
144 • SELECT c.city, s.shipper_name, SUM(oi.quantity * oi.price_each) AS revenue
145 FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id
146 JOIN Shippers s ON o.shipper_id = s.shipper_id JOIN Order_Items oi ON o.order_id = oi.order_id
147 GROUP BY c.city, s.shipper_name ORDER BY revenue DESC LIMIT 3;
```

Result Grid

	city	shipper_name	revenue
▶	Delhi	DTDC Courier	66000.00
	Noida	Delhivery Logistics	56000.00
	Jaipur	BlueDart Express	55000.00

23. List all orders with customer name, product name, supplier, and shipper.

```
148
149 • SELECT o.order_id, c.name AS customer_name, p.product_name, s.supplier_name, sh.shipper_name
150 FROM Orders o JOIN Customers c ON o.customer_id = c.customer_id
151 JOIN Order_Items oi ON o.order_id = oi.order_id
152 JOIN Products p ON oi.product_id = p.product_id
153 JOIN Suppliers s ON p.supplier_id = s.supplier_id
154 JOIN Shippers sh ON o.shipper_id = sh.shipper_id;
155
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	order_id	customer_name	product_name	supplier_name	shipper_name
▶	1	Rohit Kumar	Iphone 15	Reliance Digital	Delhivery Logistics
	12	Megha Kapoor	Iphone 15	Reliance Digital	India Post Speed
	21	Ritika Gupta	Iphone 15	Reliance Digital	Ekart Logistics
	31	Ritika Gupta	Iphone 15	Reliance Digital	DTDC Courier
	41	Rohit Kumar	Iphone 15	Reliance Digital	India Post Speed
	50	Ananya Das	Iphone 15	Reliance Digital	DTDC Courier
	61	Amit Patel	Iphone 15	Reliance Digital	BlueDart Express
	70	Aditya Jain	Iphone 15	Reliance Digital	DTDC Courier
	4	Priya Reddy	Samsung Galaxy	Croma Electronics	BlueDart Express
	11	Suresh Iyer	Samsung Galaxy	Croma Electronics	BlueDart Express
	22	Rohit Kumar	Samsung Galaxy	Croma Electronics	DTDC Courier
	32	Arjun Mehta	Samsung Galaxy	Croma Electronics	Gati Express
	40	Megha Kapoor	Samsung Galaxy	Croma Electronics	BlueDart Express
	52	Neha Verma	Samsung Galaxy	Croma Electronics	Shadowfax Delivery

Result 61

24. Show total sales per supplier along with average order value.

```
155
156 • SELECT s.supplier_name, SUM(oi.quantity * oi.price_each) AS total_sales,
157 AVG(oi.quantity * oi.price_each) AS avg_order_value FROM Suppliers s
158 JOIN Products p ON s.supplier_id = p.supplier_id JOIN Order_Items oi ON p.product_id = oi.product_id
159 GROUP BY s.supplier_name;
160
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	supplier_name	total_sales	avg_order_value
▶	Reliance Digital	240000.00	30000.000000
	Croma Electronics	307600.00	20506.666667
	Flipkart Seller Hub	96000.00	13714.285714
	Amazon India Vendor	104000.00	13000.000000
	Tata Cliq	27000.00	3375.000000
	Vijay Sales	35000.00	5000.000000
	Lifestyle Stores	70500.00	4406.250000
	Metro Electronics	77000.00	12833.333333

25.Show product categories that contributed more than 30% of total sales revenue.

160

161 •

SELECT p.category, SUM(oi.quantity * oi.price_each) AS category_sales

FROM Products p JOIN Order_Items oi ON p.product_id = oi.product_id

GROUP BY p.category HAVING SUM(oi.quantity * oi.price_each) >

0.30 * (SELECT SUM(quantity * price_each) FROM Order_Items);

165

<

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

IA

	category	category_sales
▶	Electronics	582600.00

▼



Thank You!

